

# Documentation technique - Pipeline DevSecOps

Stagiaire: Mohamed Habib Baili

Encadrant: Baylassen

Entreprise: L2T

Projet: Krayin Laravel CRM

Date: 2026-06-28

# Documentation technique - Pipeline DevSecOps

## 1. Contexte

Le projet utilise Krayin CRM, une application Laravel 12 basée sur PHP 8.3, Pest, Vite et MySQL. Le code métier principal est organisé sous `packages/Webkul`, avec des modules comme Admin, Lead, Contact, Product, Quote, User, Core et DataGrid.

L'objectif de la pipeline est d'automatiser les contrôles de qualité, les tests et les vérifications de sécurité à chaque modification du code.

## 2. Architecture applicative

Le fonctionnement général est le suivant:

1. L'utilisateur accède à une route Laravel, par exemple `/admin/dashboard`.
2. Le provider `Webkul\Admin\Providers\AdminServiceProvider` charge les routes admin.
3. La route appelle un contrôleur, par exemple `LeadController`.
4. Le contrôleur délègue la logique métier à un repository, par exemple `LeadRepository`.
5. Le repository manipule les modèles Eloquent, par exemple `Lead`, `Person`, `Product`.
6. Les migrations créent et mettent à jour les tables MySQL.
7. Les vues Blade et les assets Vite affichent l'interface.

## 3. Pipeline CI/CD proposée

La pipeline est définie dans `.github/workflows/devsecops-pipeline.yml`.

### Étape 1 - Récupération et intégrité du code

- Checkout du dépôt.
- Vérification `git status`.
- Vérification `git fsck`.

But: confirmer que le dépôt est récupéré correctement et que son historique Git n'est pas corrompu.

### Étape 2 - Build

- Installation des dépendances PHP avec Composer.
- Installation des dépendances JavaScript avec npm.
- Build frontend avec `npm run build`.

But: confirmer que l'application peut être générée sans erreur.

### Étape 3 - Qualité du code

- Vérification du style PHP avec Laravel Pint.
- Analyse SonarQube si `SONAR_HOST_URL` et `SONAR_TOKEN` sont configurés.

SonarQube analyse:

- Bugs
- Code smells
- Duplications
- Maintenabilité
- Couverture de tests

### Étape 4 - Tests unitaires et fonctionnels

- Installation de la base de test Krayin.
- Exécution des tests Pest.
- Génération d'un rapport JUnit.

- Génération d'une couverture Clover pour SonarQube.

Commande centrale:

```
php artisan test --compact --coverage-clover=coverage/clover.xml --log-junit=reports/tests/junit.xml
```

## Étape 5 - SAST, secrets et dépendances

Contrôles intégrés:

- composer audit pour les dépendances PHP.
- npm audit pour les dépendances JavaScript.
- Gitleaks pour détecter les secrets exposés.
- Semgrep pour l'analyse statique PHP.
- Trivy FS pour scanner les dépendances et fichiers du dépôt.

## Étape 6 - Artefacts

La pipeline publie:

- Rapport JUnit
- Couverture Clover
- Rapport Composer Audit
- Rapport npm Audit
- Rapport Trivy SARIF
- Rapport OWASP ZAP
- Manifest de build Vite

## Étape 7 - Déploiement de test

La job DAST installe l'application dans un environnement GitHub Actions, démarre Laravel avec:

```
php artisan serve --host=127.0.0.1 --port=8000
```

Puis elle attend que l'application réponde.

## Étape 8 - DAST

OWASP ZAP Baseline scanne l'application en HTTP.

Objectif:

- Vérifier les headers de sécurité.
- Détecter les mauvaises configurations.
- Identifier des faiblesses visibles dynamiquement.

# 4. Stratégie de sécurité

La pipeline couvre plusieurs familles de risques:

```
| Risque | Contrôle |
|---|---|
| Secrets exposés | Gitleaks |
| Dépendances vulnérables | Composer Audit, npm Audit, Trivy |
| Bugs de sécurité dans le code | Semgrep |
| Mauvaise configuration HTTP | OWASP ZAP |
| Régressions applicatives | Pest |
| Mauvaise qualité ou dette technique | Pint, SonarQube |
```

# 5. Utilisation locale

Commandes à exécuter avant de pousser:

```
composer install
npm install
php artisan migrate
./vendor/bin/pint --test
npm run build
php artisan test --compact
```

```
composer audit  
npm audit
```

## 6. Limites connues

- SonarQube nécessite un serveur SonarQube ou SonarCloud et un token.
- Les tests DAST de type injection SQL, replay attack ou privilege escalation nécessitent des scénarios fonctionnels plus avancés que le scan baseline.
- Sur Windows/WAMP, Laravel doit pouvoir écrire dans storage et bootstrap/cache.